

Uninformed Search

Lecture 5

CS 4880

Kirk Duran

Today

- ▶ Why search is a legitimate computational model of intelligent problem solving.
- ▶ How to formulate a problem using states, actions, transitions, and goals.
- ▶ How search trees, frontiers, explored sets, and parent pointers organize exploration.
- ▶ How Breadth-First Search, Depth-First Search, and Iterative Deepening differ.
- ▶ How autonomous Micromouse robots turn an unknown maze into a search problem.

Key idea

Uninformed search asks how an agent can reason systematically about possible futures without any estimate of which unexplored state is most promising.

Part 1: Search as Intelligent Problem Solving

When Reflex Behavior Is Not Enough

Reflex agent

percept $\rightarrow$ immediate action

- ▶ Many tasks require a sequence of actions whose consequences extend beyond the current percept.
- ▶ The correct first action may only become apparent after considering what can happen later.

Problem-solving agent

current state $\rightarrow$ possible futures $\rightarrow$ plan

Image Placeholder

search-reasons-about-futures.png

Key idea

Search allows an agent to reason about states it has not yet physically encountered.

Problem Abstraction Removes Irrelevant Detail

Definition

A **problem formulation** is an abstract mathematical model of a task that retains the information needed to find a solution.

Physical navigation problem

Traffic, weather, buildings, passengers, road geometry, fuel, and many other details may exist in the real environment.

Search abstraction

- ▶ State → current location
- ▶ Action → move to an adjacent location
- ▶ Goal → destination

Key idea

The search algorithm operates on the abstraction, not on every physical detail of the world.

Problem-Solving Agents Treat States as Atomic

- ▶ For uninformed search, a state can be treated as a black box with a unique identity.
- ▶ The algorithm needs to know which actions are available, which successor states result, and whether a goal has been reached.
- ▶ It does not require semantic understanding of the internal meaning of a state.

Domain	Possible meaning of a state
Navigation	current location
Sliding puzzle	tile configuration
Theorem proving	partially constructed proof
Robotics	current discrete configuration

Key idea

The same search algorithm can operate across domains because the representation supplies the meaning.

Newell and Simon: Reasoning as Search

- ▶ Newell and Simon's early AI work treated problem solving as search through a symbolic **problem space**.
- ▶ Their Logic Theorist represented theorem proving using states, operators, and goals.

Logic Theorist, 1956

- ▶ States: partially constructed proofs
- ▶ Actions: legal inference rules
- ▶ Goal: derive the target theorem

Key idea

The claim was substantive: important forms of reasoning might be implemented as systematic search.

Image Placeholder

logic-theorist-problem-space.png

Part 2: Formalizing an Uninformed Search Problem



Our Search Model Today

State space The states relevant to the problem.

Initial state Where the agent begins.

Goal test Determines whether a state satisfies the objective.

Actions $ACTIONS(s)$ returns legal actions in state s .

Transition model $RESULT(s, a)$ returns the state produced by action a .

Uniform-step assumption

- ▶ Every legal action counts as one step.
- ▶ We ignore variable action costs.
- ▶ A shortest solution has the fewest actions.

Not today

Uniform-Cost Search, Dijkstra's algorithm, and weighted path costs.

A Solution Is a Sequence of Actions

Definition

A **solution** is an action sequence that transforms the initial state into a goal state.

Example

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_g$$

- ▶ Solution: $[a_1, a_2, a_3]$
- ▶ Solution depth: 3

Image Placeholder

solution-action-sequence.png

Key idea

In today's model, a shortest solution is a goal path of minimum depth.

State-Space Graph vs. Search Tree

State-space graph

Represents the underlying problem domain.

- ▶ vertices = states;
- ▶ edges = legal transitions.

- ▶ Multiple search-tree nodes can correspond to the same state-space vertex.
- ▶ The tree records paths being considered, not a second physical world.

Search tree

Generated dynamically by the algorithm.

- ▶ root = initial state;
- ▶ children = generated successors.

Key idea

The problem defines the graph; the algorithm generates only the portion of the search tree it needs.

Why Generate States On Demand?

- ▶ Many search spaces are too large to enumerate in advance.
- ▶ 8-puzzle permutations: $9! = 362,880$.
- ▶ 15-puzzle permutations: $16! \approx 2.09 \times 10^{13}$.
- ▶ More realistic AI problems can have vastly larger or effectively unbounded spaces.

Key idea

Generate successors only when a node is expanded.

Image Placeholder

`generate-search-space-on-demand.png`

Frontier and Explored Set

Frontier

Nodes discovered but not yet expanded.

- ▶ boundary between explored and unexplored possibilities;
- ▶ strategy determines which frontier node is expanded next.

Explored set

States that have already been expanded.

- ▶ avoids unnecessary repeated exploration;
- ▶ especially important when the graph contains cycles.

Key idea

BFS and DFS use the same basic search machinery; they differ primarily in how the frontier is organized.

Parent Pointers Turn a Goal into a Plan

- ▶ Each newly generated search node can store the node that generated it.
- ▶ When a goal is found, follow parent pointers backward to the initial node.
- ▶ Reverse the sequence to recover the forward solution path.

Search-node information

state	parent	action
-------	--------	--------

Image Placeholder

parent-pointers-reconstruct-path.png

Key idea

Search discovers a goal; parent information reconstructs the action sequence that reaches it.

How Should We Evaluate a Search Strategy?

Completeness Is a solution guaranteed to be found when one exists?

Shortest-solution guarantee Is the returned solution guaranteed to have minimum depth?

Time complexity How many search nodes may be generated or expanded?

Space complexity How many search nodes must be retained simultaneously?

Notation

b = branching factor, d = depth of the shallowest goal, m = maximum search depth.

Part 3: Breadth-First Search

Breadth-First Search Expands by Depth

Definition

Breadth-First Search (BFS) expands the shallowest unexpanded node first.

- ▶ Explore depth 0, then depth 1, then depth 2, and so on.
- ▶ The frontier is maintained as a FIFO queue.
- ▶ First discovered → first expanded.

Image Placeholder

bfs-level-order.png

Breadth-First Search: General Structure

Pseudocode

```
BFS(initial_state, goal_test):  
  frontier ← FIFO queue containing initial_state  
  explored ← ∅  
  while frontier is not empty:  
    node ← frontier.dequeue()  
    if goal_test(node.state): return solution(node)  
    add node.state to explored  
    for each action in ACTIONS(node.state):  
      child ← RESULT(node.state, action)  
      if child is new: frontier.enqueue(child)  
  return failure
```

Key idea

FIFO ordering is what turns the generic search procedure into breadth-first exploration.

BFS: Guarantees and Cost

Why minimum depth?

If the shallowest goal is at depth d , BFS finishes depths $0, 1, \dots, d - 1$ before exploring anything deeper than d .

- ▶ Complete: **Yes**, for finite branching factor.
- ▶ Minimum-depth solution: **Yes**.

Complexity

- ▶ Time: $O(b^d)$
- ▶ Space: $O(b^d)$
- ▶ The frontier retains a broad layer of alternatives.
- ▶ Memory often becomes BFS's practical limiting resource.

Key idea

BFS buys systematic minimum-depth exploration with exponential frontier growth.

Part 4: Depth-First and Iterative Deepening Search



Depth-First Search Commits to One Branch

Definition

Depth-First Search (DFS) expands the deepest unexpanded node first.

- ▶ Follow one branch as deeply as possible.
- ▶ Backtrack when the branch cannot continue productively.
- ▶ The frontier is maintained as a LIFO stack or through recursion.

Image Placeholder

`dfs-deep-branch-backtrack.png`

Depth-First Search: General Structure

Pseudocode

```
DFS(initial_state, goal_test):  
    frontier ← LIFO stack containing initial_state  
    explored ← ∅  
    while frontier is not empty:  
        node ← frontier.pop()  
        if goal_test(node.state): return solution(node)  
        add node.state to explored  
        for each action in ACTIONS(node.state):  
            child ← RESULT(node.state, action)  
            if child is new: frontier.push(child)  
    return failure
```

Key idea

The generic structure is nearly identical to BFS; frontier discipline changes the behavior dramatically.

DFS Trades Guarantees for Memory

Properties

- ▶ Complete: **No**, in general.
- ▶ Minimum-depth solution: **No**.
- ▶ Time: $O(b^m)$.
- ▶ Space: approximately $O(m)$ in the simplified tree-search model.

Failure modes

- ▶ infinite or very deep branch;
- ▶ deep goal found before shallow goal;
- ▶ strong sensitivity to successor ordering.

Key idea

DFS keeps little active search state, but it can spend substantial effort in an unproductive region.

Depth Limits Lead to Iterative Deepening

Depth-Limited Search

DFS with a maximum depth ℓ .

- ▶ prevents uncontrolled descent;
- ▶ fails if every solution lies deeper than ℓ .

IDDFS

Repeat depth-limited DFS with:

$$\ell = 0, 1, 2, 3, \dots$$

until a goal is found.

Key idea

Iterative deepening searches increasingly deep prefixes without committing BFS-level memory.

IDDFS Combines Useful Properties of BFS and DFS

Properties

- ▶ Complete: **Yes**.
- ▶ Minimum-depth solution: **Yes**.
- ▶ Time: $O(b^d)$.
- ▶ Space: $O(d)$ in the simplified presentation.

Why repeated work is acceptable

Nodes near the root are expanded several times, but exponentially growing trees contain most nodes near the deepest explored level.

Key idea

IDDFS accepts modest repeated computation to obtain BFS-like depth guarantees with DFS-like memory usage.

Three Frontier Policies

Strategy	Expand next	Complete?	Min depth?	Tradeoff
BFS	shallowest	Yes	Yes	high memory
DFS	deepest	No*	No	low memory
IDDFS	deepest within limit	Yes	Yes	repeated work

*DFS can be complete on appropriately finite graph-search formulations, but not in the general unbounded case used for comparison here.

Key idea

Uninformed strategies differ not in what the goal means, but in how they organize systematic exploration.

Part 5: Is Uninformed Search Really Artificial Intelligence?



“But Isn’t This Just an Algorithm?”

Why it feels ordinary

- ▶ BFS and DFS use queues and stacks.
- ▶ They are deterministic procedures.
- ▶ They do not learn from data.
- ▶ They are standard graph algorithms.

The better question

What capability does the algorithm provide to an agent?

- ▶ represent alternatives;
- ▶ predict consequences;
- ▶ construct a goal-directed plan.

Key idea

Algorithms and artificial intelligence are not opposites; AI systems are implemented through algorithms.

Search Is a Form of Internal Simulation

A search-based agent can ask

- ▶ What happens if I take action A ?
 - ▶ What becomes possible afterward?
 - ▶ What if I instead take action B ?
 - ▶ Does any sequence eventually reach the goal?
-
- ▶ The agent need not physically execute every alternative.
 - ▶ The transition model predicts consequences inside the representation.

Image Placeholder

search-inside-intelligent-agent.png

Key idea

Deliberation separates considering an action from executing it.

Why This Was Legitimately AI

- ▶ Newell and Simon proposed that human problem solving could be modeled as symbolic manipulation in a problem space.
- ▶ Logic Theorist demonstrated that theorem proving could be implemented as search over partial proofs.
- ▶ The scientific hypothesis was not that “graphs are intelligent.”
- ▶ It was that important forms of reasoning could emerge from representation, operators, goals, and systematic exploration.

Agent-level interpretation

environment → representation → possible futures → search → plan → action

Key idea

Search belongs in AI because it operationalizes a theory of goal-directed reasoning.

Why Search Feels Less Like AI Today

- ▶ Techniques often stop feeling like AI once they become well understood and widely standardized.
- ▶ Search is now taught in algorithms courses because its behavior can be characterized precisely.
- ▶ Familiarity does not erase the role it plays inside autonomous reasoning systems.

Important distinction

BFS by itself is not an intelligent agent. An autonomous agent that uses a state-space model and BFS to construct a goal-directed plan is performing a foundational form of AI problem solving.

Key idea

The algorithm may be ordinary; the capability it enables—reasoning about alternatives before acting—is fundamental.

Part 6: Application — Autonomous Micromouse

Application: The Micromouse Competition

- ▶ Micromouse is an autonomous robotics competition centered on maze discovery and navigation.
- ▶ The robot carries its own computation, sensing, actuation, and power.
- ▶ It does not begin with a complete map of the competition maze.
- ▶ It must discover enough structure to reach the goal and exploit what it learns.

AI question

How can an autonomous agent construct a useful route when its model of the environment is initially incomplete?

Image Placeholder

micromouse-competition-overview.png

Abstract the Robot into a Search Problem

State Current maze cell.

Initial state Starting cell near a corner.

Goal Reach a designated center goal cell.

Actions Move north, south, east, or west when a passage is open.

Transition Move to the adjacent cell in the selected direction.

Image Placeholder

`micromouse-discrete-search-abstraction.png`

Key idea

For this lecture, the physical robot becomes an agent moving among discrete states in a graph.

The Maze Is a Graph the Robot Must Discover

Maze → graph

- ▶ cell → state;
- ▶ open passage → transition;
- ▶ wall → unavailable action;
- ▶ center → goal.

But the graph is initially incomplete

- ▶ onboard sensors reveal nearby walls;
- ▶ the internal map is updated incrementally;
- ▶ newly discovered walls remove assumed transitions.

sense → update map → search → move → sense again

Wall Following Is Primarily Reactive

- ▶ A simple strategy is to keep one side of the robot adjacent to a wall.
- ▶ The next action depends mainly on immediately perceived wall geometry.
- ▶ This requires little explicit reasoning over alternative future routes.
- ▶ Certain maze structures can defeat wall-following as a general strategy for reaching the intended goal.

Agent interpretation

percept → local rule → action

Image Placeholder
`micromouse-wall-following-reactive.png`

Key idea

Micromouse gives us a physical contrast between reflex behavior and model-based deliberation.

Micromouse as Depth-First Search

1. At a junction, choose an unexplored branch.
2. Follow it as deeply as possible.
3. At a dead end, backtrack to the most recent junction with another option.
4. Continue until a goal is found.

Limitation

The first discovered goal may use many more moves than another route that DFS has not explored yet.

Image Placeholder

`micromouse-dfs-misses-shortcut.png`

Micromouse as Breadth-First Search

- ▶ BFS explores reachable cells in order of increasing number of moves from the start.
- ▶ In our uniform-step abstraction, the first goal found is a minimum-move route.
- ▶ Once a maze graph is known, BFS is therefore a natural shortest-depth planner.

Search computation

Expand states in memory and organize the frontier by depth.

Physical exploration

Driving down a corridor, returning, and exploring another branch requires real motion.

Key idea

The order in which a search algorithm expands states does not have to equal the physical trajectory of the robot.

Flood Fill: A Practical Micromouse Strategy

- ▶ A common classical Micromouse strategy is flood fill.
- ▶ Goal cells receive distance value 0.
- ▶ Increasing values are propagated outward through traversable neighbors.
- ▶ The mouse moves toward a neighboring cell with a smaller value.

Interpretation

The labels encode graph distance to the goal under the robot's current map.

Image Placeholder

`micromouse-flood-fill-distance-map.png`

Flood Fill Revises the Plan as Walls Are Discovered

optimistic map → encounter wall → record wall → recompute → continue

- ▶ Unknown passages can initially be treated optimistically as traversable.
- ▶ A sensed wall invalidates an assumed transition in the internal graph.
- ▶ Distance labels are updated using the revised connectivity.
- ▶ The mouse continues following decreasing values toward the goal.

Key idea

The agent repeatedly combines perception, representation, and search rather than solving the problem only once.

Micromouse Is More Than “Run BFS on a Graph”

- ▶ **Perception:** detect local walls.
- ▶ **Representation:** maintain an internal map.
- ▶ **Memory:** retain discovered structure.
- ▶ **Search:** reason over routes in the current map.
- ▶ **Planning:** select an intended movement sequence.
- ▶ **Action:** execute movement through the physical maze.

Image Placeholder

`micromouse-agent-loop.png`

Key idea

The search algorithm is one component; the intelligent system is the autonomous agent that uses representation and search to act.

What We Deliberately Ignore Today

- ▶ velocity and acceleration;
- ▶ braking distance;
- ▶ wheel dynamics and slip;
- ▶ turning radius.
- ▶ localization and sensor error;
- ▶ different execution times for turns and straight segments;
- ▶ continuous or diagonal motion.

Today's abstraction

One legal move between adjacent cells = one action.

Key idea

Today we ask which sequence of discrete states reaches the goal. The physical question of how quickly the robot can execute that route comes next.

Uninformed Search: Complete Mental Model

The agent knows

- ▶ initial/current state;
- ▶ available actions;
- ▶ transition model;
- ▶ goal test.

The agent does not know

- ▶ which unexplored state is promising;
- ▶ which branch appears closer to the goal;
- ▶ which successor deserves domain-specific preference.

environment → abstraction → state space → search → plan → action

Key idea

Search is not AI because it is complicated; it is AI because it provides a computational mechanism for goal-directed deliberation.

Exit Ticket: Choose the Search Strategy

In class

For each situation, choose BFS, DFS, or IDDFS and justify your choice using completeness, minimum solution depth, and memory requirements.

1. The goal is known to be shallow and memory is plentiful.
2. The state space may be extremely deep and memory is severely constrained.
3. The shallowest goal is important, but its depth is unknown and BFS memory is unacceptable.

Key idea

Outcome: you can formulate an uninformed search problem, explain how frontier policy changes exploration, and defend search as a foundational AI mechanism.

Next Time

- ▶ The discrete search model tells us *where* the agent should go.
- ▶ A physical robot must also determine *how* to execute that route.
- ▶ Orientation, turning, motion constraints, sensing, and execution time can change which route is practically best.

Key idea

Next: move from abstract state transitions to physical motion and richer planning constraints.

Image Placeholder Index

- ▶ search-reasons-about-futures.png
- ▶ logic-theorist-problem-space.png
- ▶ solution-action-sequence.png
- ▶ generate-search-space-on-demand.png
- ▶ parent-pointers-reconstruct-path.png
- ▶ bfs-level-order.png
- ▶ dfs-deep-branch-backtrack.png
- ▶ search-inside-intelligent-agent.png
- ▶ micromouse-competition-overview.png
- ▶ micromouse-discrete-search-abstraction.png
- ▶ micromouse-wall-following-reactive.png
- ▶ micromouse-dfs-misses-shortcut.png
- ▶ micromouse-flood-fill-distance-map.png
- ▶ micromouse-agent-loop.png

Example

Save each image under `../assets/lecture-05/`. The labeled placeholder will be replaced automatically when its filename matches.